ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH

Bài tập – Nhóm 4

Phân loại ảnh với các mô hình học sâu cơ bản

GVHD: TS. Lê Thành Sách
SV thực hiện: Nguyễn Hữu Trưởng - 2470573
Nguyễn Nhật Thanh - 2570316
Võ Thị Bích Phượng - 2470570

Tp. Hồ Chí Minh, Tháng 03/2026

Mục lục

1 Giới thiệu **4**
1.1 Mục tiêu bài tập . . . 4
1.2 Nội dung bài tập . . . 4
1.3 Tập dữ liệu và Công cụ . . . 4
1.4 Cấu trúc báo cáo . . . 4

2 Tập dữ liệu CIFAR-10 và Tiền xử lý **5**
2.1 Giới thiệu tập dữ liệu CIFAR-10 . . . 5
2.2 Phân chia dữ liệu . . . 5
2.2.1 Cho các mô hình Softmax, MLP, CNN, ViT và các biến thể . . . 5
2.2.2 Cho các mô hình LSTM/GRU . . . 5
2.3 Tiền xử lý dữ liệu (Data Preprocessing) . . . 5
2.3.1 Bộ transform cho mạng tự xây dựng (32×32) . . . 5
2.3.2 Bộ transform cho mô hình LSTM/GRU . . . 6
2.4 Hàm huấn luyện và đánh giá chung . . . 6

3 Hồi quy Softmax (Softmax Regression) **7**
3.1 Giới thiệu . . . 7
3.2 Kiến trúc và Công thức toán học . . . 7
3.2.1 Mô hình tuyến tính . . . 7
3.2.2 Hàm Softmax . . . 7
3.2.3 Hàm mất mát Cross-Entropy . . . 7
3.3 Đặc điểm của mô hình . . . 8
3.4 Áp dụng cho bài tập . . . 8
3.4.1 Kiến trúc mô hình . . . 8
3.4.2 Mã nguồn . . . 8
3.4.3 Cấu hình huấn luyện . . . 8
3.5 Kết quả thực nghiệm . . . 8
3.6 Nhận xét . . . 9

4 Mạng Perceptron Đa tầng (MLP) **9**
4.1 Giới thiệu . . . 9
4.2 Kiến trúc và Công thức toán học . . . 9
4.2.1 Cấu trúc MLP . . . 9
4.2.2 Hàm kích hoạt ReLU . . . 9
4.2.3 Kỹ thuật điều chuẩn (Regularization) . . . 9
4.3 Đặc điểm của mô hình . . . 10
4.4 Áp dụng cho bài tập . . . 10
4.4.1 Kiến trúc mô hình . . . 10
4.4.2 Mã nguồn . . . 10
4.4.3 Cấu hình huấn luyện . . . 10
4.5 Kết quả thực nghiệm . . . 10
4.6 Nhận xét . . . 11

5 Mạng Nơ-ron Tích chập (CNN) **11**
5.1 Giới thiệu . . . 11
5.2 Các thành phần cốt lõi . . . 11
5.2.1 Phép tích chập (Convolution) . . . 11
5.2.2 Kết nối tắt – Residual Connection . . . 12
5.2.3 Batch Normalization . . . 12
5.3 Đặc điểm của mô hình . . . 12
5.4 Áp dụng cho bài tập . . . 12
5.4.1 Kiến trúc CustomResNet . . . 12
5.4.2 Mã nguồn . . . 12
5.4.3 Cấu hình huấn luyện . . . 13

5.5 Kết quả thực nghiệm . . . 13
5.6 Nhận xét . . . 14

6 Vision Transformer (ViT) 14
6.1 Giới thiệu . . . 14
6.2 Kiến trúc và Công thức toán học . . . 14
6.2.1 Patch Embedding . . . 14
6.2.2 Class Token và Positional Encoding . . . 14
6.2.3 Multi-Head Self-Attention (MHSA) . . . 15
6.2.4 Transformer Encoder Block . . . 15
6.2.5 Classification Head . . . 15
6.3 Đặc điểm của mô hình . . . 15
6.4 Áp dụng cho bài tập . . . 15
6.4.1 Kiến trúc SimpleViT . . . 15
6.4.2 Mã nguồn . . . 15
6.4.3 Cấu hình huấn luyện . . . 16
6.5 Kết quả thực nghiệm . . . 16
6.6 Nhận xét . . . 17

7 Tự hiện thực TransformerEncoder và ViT 17
7.1 Mục tiêu . . . 17
7.2 Các thành phần tự hiện thực . . . 17
7.2.1 Custom Multi-Head Self-Attention (CustomMHSA) . . . 17
7.2.2 Custom MLP (Feed-Forward Network) . . . 18
7.2.3 Custom Transformer Block . . . 18
7.3 Mô hình Custom ViT . . . 18
7.4 Cấu hình huấn luyện . . . 19
7.5 Kết quả thực nghiệm . . . 19
7.6 So sánh Custom ViT vs PyTorch ViT . . . 19
7.7 Nhận xét . . . 19

8 Mô hình lai CNN + Transformer 20
8.1 Ý tưởng . . . 20
8.2 Kiến trúc . . . 20
8.2.1 CNN Backbone . . . 20
8.2.2 Biến H×W thành Token . . . 20
8.2.3 Transformer Encoder . . . 20
8.3 Mã nguồn . . . 20
8.4 Cấu hình huấn luyện . . . 21
8.5 Kết quả . . . 21
8.6 Nhận xét . . . 21

9 Tokenizer với Overlapping Patches 21
9.1 Ý tưởng . . . 21
9.2 Kiến trúc . . . 21
9.2.1 Overlapping Patch Embedding . . . 21
9.3 Mã nguồn . . . 21
9.4 Cấu hình huấn luyện . . . 22
9.5 Kết quả . . . 22
9.6 Nhận xét . . . 22

10 Mô hình phân loại dựa trên LSTM/GRU 22
10.1 Mạng Nơ-ron Hồi quy (RNN) . . . 22
10.1.1 Giới thiệu . . . 22
10.1.2 Vấn đề Triệt tiêu và Bùng nổ Đạo hàm . . . 23
10.2 LSTM (Long Short-Term Memory) . . . 23
10.2.1 Ý tưởng cốt lõi . . . 23

10.2.2 Công thức toán học . . . 23
10.3 GRU (Gated Recurrent Unit) . . . 23
10.3.1 Ý tưởng: Đơn giản hóa LSTM . . . 23
10.3.2 Công thức toán học . . . 23
10.4 Biểu diễn ảnh dưới dạng chuỗi: Phương pháp Row-wise . . . 24
10.5 Kiến trúc mô hình . . . 24
10.5.1 LSTM Classifier . . . 24
10.5.2 GRU Classifier . . . 24
10.6 Cấu hình huấn luyện . . . 25
10.7 Kết quả thực nghiệm . . . 25
10.8 Phân tích Ma trận Nhầm lẫn . . . 26
10.9 Nhận xét . . . 27

11 So sánh Tổng hợp các Mô hình **27**
11.1 Bảng so sánh hiệu suất . . . 27
11.2 Phân tích theo nhóm kiến trúc . . . 27
11.2.1 Nhóm 1: Mô hình tuyến tính / Fully-Connected (Softmax, MLP) . . . 27
11.2.2 Nhóm 2: CNN . . . 27
11.2.3 Nhóm 3: Transformer-based (ViT, Custom ViT, Hybrid, Overlapping) . . . 27
11.2.4 Nhóm 4: RNN-based (LSTM, GRU) . . . 28
11.3 Biểu đồ so sánh Test Accuracy . . . 28
11.4 Nhận xét tổng quát . . . 28

12 Nhận xét và Kết luận **28**
12.1 Nhận xét cho từng mô hình . . . 28
12.1.1 Softmax Regression . . . 28
12.1.2 MLP . . . 29
12.1.3 CNN (CustomResNet) . . . 29
12.1.4 ViT (Vision Transformer) . . . 29
12.1.5 Custom ViT (Tự hiện thực TransformerEncoder) . . . 29
12.1.6 Hybrid CNN-Transformer . . . 29
12.1.7 Overlapping Patches ViT . . . 29
12.1.8 LSTM và GRU . . . 29
12.2 Bài học rút ra . . . 29

Tài liệu tham khảo **29**

1 Giới thiệu

1.1 Mục tiêu bài tập

Bài tập này thuộc môn **Học sâu và ứng dụng trong thị giác máy tính** (mã môn học: CO5085), năm học 2025–2026, Học kỳ 2. Mục tiêu chính là giúp sinh viên nắm vững **cơ bản về học sâu và ứng dụng trong thị giác máy tính** thông qua việc:

- Xây dựng và so sánh các mô hình phân loại ảnh từ đơn giản đến phức tạp: **Softmax Regression**, **MLP**, **CNN**, **Vision Transformer (ViT)**, và mô hình dựa trên **LSTM/GRU**.
- Tự viết vòng lặp huấn luyện (training loop) bằng PyTorch, sử dụng `backward` có sẵn để tính đạo hàm, thay vì chỉ gọi API cấp cao.
- Hiểu cấu trúc bên trong Transformer qua việc tự hiện thực khối `TransformerEncoder` từ các phép toán cơ bản.
- Khám phá các cách thiết kế kiến trúc khác nhau: kết hợp CNN–Transformer, các cách tokenize/embed ảnh khác nhau, và biểu diễn ảnh thành chuỗi cho RNN.

1.2 Nội dung bài tập

Theo yêu cầu đề bài, bài tập được chia thành 5 phần chính:

1. **Phần 1 – Xây dựng các mô hình phân loại:** Xây dựng lần lượt bốn mô hình Softmax Regression, MLP, CNN và ViT (sử dụng khối có sẵn của PyTorch) cho bài toán phân loại ảnh trên tập dữ liệu CIFAR-10.
2. **Phần 2 – Huấn luyện, đánh giá và so sánh:** Tự viết giải thuật huấn luyện (training loop) dựa trên PyTorch với vòng lặp theo epoch/batch, gọi `loss.backward()` và cập nhật tham số bằng optimizer. Sử dụng vòng lặp này để huấn luyện, đánh giá và so sánh bốn mô hình ở Phần 1.
3. **Phần 3 – Tự hiện thực TransformerEncoder và ViT:** Tự hiện thực khối `TransformerEncoder` từ các phép toán cơ bản của PyTorch (Linear, LayerNorm, einsum), *không* dùng `nn.TransformerEncoderLayer` hay `nn.TransformerEncoder` có sẵn. Xây dựng ViT dựa trên khối tự hiện thực và so sánh với ViT dùng khối có sẵn ở Phần 1.
4. **Phần 4 – Các kiến trúc kết hợp và cách embed ảnh khác nhau:** Xây dựng các kiến trúc lai như **Hybrid CNN + Transformer** (dùng CNN làm backbone trích đặc trưng rồi đưa vào Transformer) và **Overlapping Patches ViT** (dùng patch chồng mép để tạo token).
5. **Phần 5 – Mô hình phân loại dựa trên LSTM/GRU:** Xây dựng mô hình phân loại dựa trên LSTM và GRU, tự chọn cách biểu diễn ảnh thành chuỗi (sequence). So sánh LSTM vs GRU trên cùng cách biểu diễn.

1.3 Tập dữ liệu và Công cụ

Nhóm lựa chọn tập dữ liệu **CIFAR-10** gồm 60.000 ảnh màu RGB kích thước 32×32 pixel, phân bổ đều vào 10 lớp. Toàn bộ thực nghiệm được hiện thực bằng **PyTorch** và huấn luyện trên GPU NVIDIA Tesla T4.

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức theo thứ tự từ mô hình đơn giản đến phức tạp: Softmax Regression $\rightarrow$ MLP $\rightarrow$ CNN $\rightarrow$ ViT $\rightarrow$ Custom ViT $\rightarrow$ Hybrid CNN-Transformer $\rightarrow$ Overlapping ViT $\rightarrow$ LSTM/GRU. Mỗi phần bao gồm: giới thiệu lý thuyết, kiến trúc mô hình, công thức toán học, mã nguồn hiện thực, cấu hình huấn luyện, và kết quả thực nghiệm. Cuối cùng là phần so sánh tổng hợp và nhận xét.

2 Tập dữ liệu CIFAR-10 và Tiền xử lý

2.1 Giới thiệu tập dữ liệu CIFAR-10

CIFAR-10 (Canadian Institute For Advanced Research) [1] là một trong những tập dữ liệu chuẩn (benchmark) phổ biến nhất trong lĩnh vực thị giác máy tính. Tập dữ liệu bao gồm **60.000 ảnh màu RGB** có kích thước 32×32 pixel, phân bổ **đều** vào 10 lớp (mỗi lớp 6.000 ảnh):

airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

Tập dữ liệu được chia sẵn thành 50.000 ảnh huấn luyện (training) và 10.000 ảnh kiểm thử (test). Phân bố các lớp hoàn toàn cân bằng với 5.000 ảnh mỗi lớp trong tập huấn luyện.

Hình 1: *Ảnh mẫu từ 10 lớp trong tập dữ liệu CIFAR-10*

2.2 Phân chia dữ liệu

2.2.1 Cho các mô hình Softmax, MLP, CNN, ViT và các biến thể

Từ tập huấn luyện gốc 50.000 ảnh, nhóm chia thành hai phần bằng `random_split` với `seed=42`:

Bảng 1: *Phân chia dữ liệu cho các mô hình CNN/ViT*

Tập dữ liệu	Số lượng	Tỷ lệ	Mục đích
Training Set	40.000	80%	Huấn luyện mô hình
Validation Set	10.000	20%	Theo dõi hiệu suất, chọn mô hình tốt nhất
Test Set	10.000	–	Đánh giá cuối cùng (tập độc lập)

2.2.2 Cho các mô hình LSTM/GRU

Do huấn luyện trên notebook riêng với cấu hình khác, tập dữ liệu được chia:

Bảng 2: *Phân chia dữ liệu cho các mô hình LSTM/GRU*

Tập dữ liệu	Số lượng	Tỷ lệ	Mục đích
Training Set	45.000	90%	Huấn luyện mô hình
Validation Set	5.000	10%	Theo dõi hiệu suất
Test Set	10.000	–	Đánh giá cuối cùng

2.3 Tiền xử lý dữ liệu (Data Preprocessing)

2.3.1 Bộ transform cho mạng tự xây dựng (32×32)

Các mô hình Softmax, MLP, CNN, ViT và các biến thể sử dụng ảnh gốc 32×32 pixel với chuẩn hóa về khoảng $[-1, 1]$. Riêng tập huấn luyện được áp dụng thêm các kỹ thuật tăng cường dữ liệu (data augmentation):

```
train_transform_custom = transforms.Compose([
    transforms.RandomRotation(15),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1),
    transforms.RandomAdjustSharpness(sharpness_factor=2, p=0.1),
    transforms.ToTensor(),
```

```
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    transforms.RandomErasing(p=0.5, scale=(0.02, 0.1), value=1.0)
])

test_transform_custom = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

Các kỹ thuật tăng cường bao gồm: xoay ngẫu nhiên (RandomRotation 15), lật ngang ngẫu nhiên (RandomHorizontalFlip), biến đổi màu sắc (ColorJitter), điều chỉnh độ nét (RandomAdjustSharpness), và xóa ngẫu nhiên (RandomErasing). Mục đích là tăng tính đa dạng của dữ liệu huấn luyện, giúp mô hình tổng quát hóa tốt hơn.

2.3.2 Bộ transform cho mô hình LSTM/GRU

Các mô hình LSTM/GRU sử dụng chuẩn hóa theo giá trị trung bình và độ lệch chuẩn thực tế của CIFAR-10, *không* áp dụng data augmentation:

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        mean=(0.4914, 0.4822, 0.4465),  # Mean cua CIFAR-10
        std=(0.2470, 0.2435, 0.2616)    # Std cua CIFAR-10
    )
])
```

2.4 Hàm huấn luyện và đánh giá chung

Nhóm xây dựng một hàm huấn luyện chung `train_and_evaluate` cho tất cả các mô hình (trừ LSTM/GRU), bao gồm các thành phần:

- **Hàm mất mát:** `CrossEntropyLoss` (đã tích hợp Softmax bên trong).
- **Thuật toán tối ưu:** Adam.
- **Scheduler:** `ReduceLROnPlateau` – giảm learning rate đi một nửa khi validation loss không cải thiện sau 2 epoch.
- **Early Stopping:** Dừng huấn luyện sớm khi validation loss không cải thiện sau một số epoch nhất định (patience), đồng thời lưu checkpoint tốt nhất.

```
def train_and_evaluate(model_name, model, train_loader, val_loader,
                       epochs, lr, device, patience=5, save_path=None):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', factor=0.5, patience=2)

    best_val_loss = float('inf')
    epochs_no_improve = 0

    for epoch in range(epochs):
        # --- TRAIN ---
        model.train()
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

```

```
        # --- VALIDATE ---
        model.eval()
        with torch.no_grad():
            # Tinh val_loss, val_acc ...
            pass

        scheduler.step(val_loss)

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            epochs_no_improve = 0
            torch.save(model.state_dict(), save_path)
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:
                break  # Early stopping

    model.load_state_dict(torch.load(save_path))
    return history
```

3 Hồi quy Softmax (Softmax Regression)

3.1 Giới thiệu

Hồi quy Softmax (Softmax Regression), còn gọi là hồi quy logistic đa lớp (Multinomial Logistic Regression), là mô hình phân loại tuyến tính đơn giản nhất cho bài toán phân loại nhiều lớp. Mô hình này ánh xạ trực tiếp vector đầu vào thành phân phối xác suất trên các lớp, *không* sử dụng bất kỳ lớp ẩn nào.

3.2 Kiến trúc và Công thức toán học

3.2.1 Mô hình tuyến tính

Cho vector đầu vào $\mathbf{x} \in \mathbb{R}^d$ (ảnh đã được flatten), mô hình tính *logits* (điểm số thô) cho mỗi lớp k:

$$z_k = \mathbf{w}_k^T \mathbf{x} + b_k, \quad k = 1, 2, \ldots, K \tag{1}$$

trong đó $\mathbf{w}_k \in \mathbb{R}^d$ là vector trọng số và b_k là hệ số chệch của lớp k, $K = 10$ là số lớp.

Viết dưới dạng ma trận: $\mathbf{z} = W\mathbf{x} + \mathbf{b}$ với $W \in \mathbb{R}^{K \times d}$, $\mathbf{b} \in \mathbb{R}^K$.

3.2.2 Hàm Softmax

Để chuyển logits thành phân phối xác suất, hàm **Softmax** được áp dụng:

$$P(y = k \mid \mathbf{x}) = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \tag{2}$$

Hàm Softmax đảm bảo: (1) mọi xác suất đều dương $P(y = k|\mathbf{x}) > 0$, và (2) tổng xác suất bằng 1: $\sum_k P(y = k|\mathbf{x}) = 1$.

3.2.3 Hàm mất mát Cross-Entropy

Hàm mất mát Cross-Entropy đo khoảng cách giữa phân phối dự đoán và nhãn thật (one-hot encoding):

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log P(y = k \mid \mathbf{x}) = -\log P(y = c \mid \mathbf{x}) \tag{3}$$

trong đó c là lớp đúng của mẫu. Trong PyTorch, `nn.CrossEntropyLoss` đã tích hợp sẵn cả hàm Softmax và Cross-Entropy, nên mô hình chỉ cần trả về logits.

3.3 Đặc điểm của mô hình

- **Ưu điểm:** Đơn giản, ít tham số, dễ hiểu, huấn luyện nhanh. Là baseline tốt để so sánh.
- **Nhược điểm:** Chỉ học được ranh giới phân loại tuyến tính (linear decision boundary), không thể nắm bắt các mối quan hệ phi tuyến hay đặc trưng không gian (spatial features) trong ảnh. Việc flatten ảnh thành vector 1D làm mất hoàn toàn cấu trúc không gian 2D.

3.4 Áp dụng cho bài tập

3.4.1 Kiến trúc mô hình

Ảnh CIFAR-10 có kích thước $3 \times 32 \times 32 = 3072$ pixel. Mô hình gồm hai bước:

1. **Flatten:** Biến ảnh $(B, 3, 32, 32)$ thành vector $(B, 3072)$.
2. **Linear:** Ánh xạ tuyến tính $\text{Linear}(3072, 10)$ cho 10 lớp.

Tổng số tham số: $3072 \times 10 + 10 = 30.730$

3.4.2 Mã nguồn

```python
class SoftmaxRegression(nn.Module):
    def __init__(self):
        super(SoftmaxRegression, self).__init__()
        self.flatten = nn.Flatten()
        self.linear = nn.Linear(in_features=32*32*3, out_features=10)

    def forward(self, x):
        x = self.flatten(x)        # (B, 3, 32, 32) -> (B, 3072)
        logits = self.linear(x)    # (B, 3072) -> (B, 10)
        return logits
```

3.4.3 Cấu hình huấn luyện

Bảng 3: *Cấu hình huấn luyện Softmax Regression*

Thông số	Giá trị
Optimizer	Adam (weight_decay=10^{-4})
Learning Rate	10^{-3}
Batch Size	512
Max Epochs	100
Early Stopping Patience	5
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=2)
Data Augmentation	Có (transform_custom)

3.5 Kết quả thực nghiệm

Mô hình dừng huấn luyện sớm tại **Epoch 24** do early stopping. Learning rate được giảm tự động từ 10^{-3} xuống 1.25×10^{-4} qua nhiều lần kích hoạt scheduler.

Bảng 4: *Kết quả huấn luyện Softmax Regression qua các epoch đáng chú ý*

Epoch	LR	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	10^{-3}	1.9177	32.26	1.8420	35.64
10	10^{-3}	1.7955	38.02	1.7916	37.92
14	5×10^{-4}	1.7776	38.88	1.7734	39.20
19	2.5×10^{-4}	1.7592	39.74	**1.7565**	39.90
24	1.25×10^{-4}	1.7520	40.19	1.7569	**40.04**

- **Best Validation Loss:** 1.7565 tại Epoch 19.

- **Best Validation Accuracy:** 40.04% tại Epoch 24.
- **Test Accuracy: 40.89%**.

3.6 Nhận xét

Với Test Accuracy chỉ đạt 40.89% (gần ngẫu nhiên là 10% cho 10 lớp), mô hình Softmax Regression cho thấy những hạn chế rõ ràng:

- Ranh giới phân loại tuyến tính không đủ phức tạp để phân biệt 10 lớp ảnh có sự tương đồng cao (ví dụ: cat vs dog, automobile vs truck).
- Việc flatten ảnh thành vector 1D phá hủy hoàn toàn cấu trúc không gian, khiến mô hình không thể khai thác thông tin về vị trí tương đối giữa các pixel.
- Khoảng cách giữa Train Accuracy (40.19%) và Val Accuracy (40.04%) rất nhỏ, cho thấy mô hình *không* bị quá khớp mà đơn giản là *thiếu năng lực biểu diễn* (underfitting).

4 Mạng Perceptron Đa tầng (MLP)

4.1 Giới thiệu

Mạng Perceptron Đa tầng (Multi-Layer Perceptron – MLP) là sự mở rộng tự nhiên của Softmax Regression bằng cách thêm một hoặc nhiều *lớp ẩn* (hidden layers) với hàm kích hoạt phi tuyến. Nhờ đó, MLP có khả năng học các ranh giới phân loại phi tuyến phức tạp hơn.

Theo **Định lý Xấp xỉ Tổng quát** (Universal Approximation Theorem) [11], một MLP chỉ với một lớp ẩn đủ rộng có thể xấp xỉ bất kỳ hàm liên tục nào với độ chính xác tùy ý. Tuy nhiên trên thực tế, mạng sâu hơn (nhiều lớp hẹp hơn) thường hiệu quả hơn mạng nông rộng.

4.2 Kiến trúc và Công thức toán học

4.2.1 Cấu trúc MLP

Một MLP gồm L lớp ẩn thực hiện chuỗi biến đổi:

$$\mathbf{h}^{(l)} = \phi\left(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, 2, \ldots, L \tag{4}$$

trong đó $\mathbf{h}^{(0)} = \mathbf{x}$ là đầu vào (ảnh đã flatten), $W^{(l)}$ và $\mathbf{b}^{(l)}$ là trọng số và bias của lớp l, ϕ là hàm kích hoạt phi tuyến.

Lớp đầu ra tính logits: $\mathbf{z} = W^{(L+1)}\mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}$

4.2.2 Hàm kích hoạt ReLU

Hàm kích hoạt **ReLU** (Rectified Linear Unit) được sử dụng phổ biến nhất:

$$\mathrm{ReLU}(x) = \max(0, x) \tag{5}$$

ReLU đơn giản, tính toán nhanh, và giúp giảm thiểu vấn đề triệt tiêu đạo hàm so với sigmoid hay tanh.

4.2.3 Kỹ thuật điều chuẩn (Regularization)

Để chống quá khớp, nhóm sử dụng hai kỹ thuật:

Dropout [9]: Tắt ngẫu nhiên một tỷ lệ p các neuron trong quá trình huấn luyện, buộc mạng không phụ thuộc vào bất kỳ neuron đơn lẻ nào:

$$\hat{\mathbf{h}}^{(l)} = \mathbf{m}^{(l)} \odot \mathbf{h}^{(l)}, \quad m_j^{(l)} \sim \mathrm{Bernoulli}(1-p) \tag{6}$$

Batch Normalization [10]: Chuẩn hóa đầu ra của mỗi lớp theo mini-batch, giúp ổn định và tăng tốc quá trình huấn luyện:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma\hat{x}_i + \beta \tag{7}$$

trong đó μ_B, σ_B^2 là trung bình và phương sai của mini-batch, γ, β là tham số học được.

4.3 Đặc điểm của mô hình

- **Ưu điểm:** Có khả năng học biểu diễn phi tuyến, linh hoạt về kiến trúc, dễ mở rộng.
- **Nhược điểm:** Vẫn yêu cầu flatten ảnh thành vector 1D nên mất cấu trúc không gian. Số lượng tham số lớn do kết nối đầy đủ (fully connected). Dễ bị quá khớp trên tập dữ liệu nhỏ.

4.4 Áp dụng cho bài tập

4.4.1 Kiến trúc mô hình

Mô hình MLP gồm hai lớp ẩn với BatchNorm, ReLU và Dropout:

$$\texttt{Flatten(3072)} \to \texttt{Linear(3072}\to\texttt{512)} \to \texttt{BN} \to \texttt{ReLU} \to \texttt{Dropout(0.3)}$$
$$\to \texttt{Linear(512}\to\texttt{128)} \to \texttt{BN} \to \texttt{ReLU} \to \texttt{Dropout(0.3)} \to \texttt{Linear(128}\to\texttt{10)}$$

4.4.2 Mã nguồn

```python
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.network = nn.Sequential(
            # Lop an 1: 3072 -> 512
            nn.Linear(in_features=32*32*3, out_features=512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(p=0.3),
            # Lop an 2: 512 -> 128
            nn.Linear(in_features=512, out_features=128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(p=0.3),
            # Lop dau ra: 128 -> 10
            nn.Linear(in_features=128, out_features=10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.network(x)
        return logits
```

4.4.3 Cấu hình huấn luyện

Tiền xử lý dữ liệu giống với Softmax Regression (sử dụng `train_transform_custom`).

Bảng 5: *Cấu hình huấn luyện MLP*

Thông số	Giá trị
Optimizer	Adam
Learning Rate	10^{-3}
Batch Size	512
Max Epochs	100
Early Stopping Patience	7
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=2)

4.5 Kết quả thực nghiệm

Mô hình MLP huấn luyện đủ **100 epoch** (không bị early stopping dừng sớm), với learning rate giảm dần từ 10^{-3} xuống 8×10^{-6}.

Bảng 6: *Kết quả huấn luyện MLP qua các epoch đáng chú ý*

Epoch	LR	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	10^{-3}	1.7762	37.24	1.6115	42.74
20	10^{-3}	1.4228	49.50	1.4100	49.80
50	6.25×10^{-5}	1.2394	55.80	1.2289	56.28
94	1.6×10^{-5}	1.2155	56.66	**1.2134**	56.98
98	8×10^{-6}	1.2055	57.09	1.2102	**57.34**
100	8×10^{-6}	1.2054	56.88	1.2223	57.06

- **Best Validation Loss:** 1.2102 tại Epoch 98.
- **Best Validation Accuracy:** 57.34%.
- **Test Accuracy: 58.91**%.

4.6 Nhận xét

MLP đạt Test Accuracy 58.91%, cao hơn Softmax Regression (40.89%) khoảng 18%, chứng tỏ các lớp ẩn phi tuyến giúp mô hình học được biểu diễn tốt hơn đáng kể. Tuy nhiên:

- Mô hình hội tụ rất chậm (cần đến 100 epoch), cho thấy việc học trực tiếp trên vector flatten 3072 chiều là không hiệu quả.
- Khoảng cách Train-Val Accuracy nhỏ ($\sim 0.2\%$), cho thấy mô hình vẫn bị *underfitting* – Dropout 0.3 và BatchNorm đủ mạnh để ngăn quá khớp, nhưng kiến trúc chưa đủ năng lực.
- Giới hạn $\sim 59\%$ là "trần" của các phương pháp dựa trên fully-connected thuần túy trên CIFAR-10, do không khai thác được đặc trưng không gian cục bộ.

5 Mạng Nơ-ron Tích chập (CNN)

5.1 Giới thiệu

Mạng Nơ-ron Tích chập (Convolutional Neural Network – CNN) là kiến trúc được thiết kế đặc biệt để xử lý dữ liệu có cấu trúc lưới (grid-like topology), đặc biệt là ảnh. CNN khắc phục nhược điểm lớn nhất của MLP: thay vì flatten ảnh và mất cấu trúc không gian, CNN sử dụng các *bộ lọc tích chập* (convolutional filters) trượt trên ảnh để trích xuất đặc trưng cục bộ (local features) [8].

5.2 Các thành phần cốt lõi

5.2.1 Phép tích chập (Convolution)

Phép tích chập 2D áp dụng bộ lọc $\mathbf{K} \in \mathbb{R}^{k \times k}$ lên ảnh đầu vào $\mathbf{X}$:

$$(\mathbf{X} * \mathbf{K})_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \mathbf{X}_{i+m,j+n} \cdot \mathbf{K}_{m,n} \tag{8}$$

Mỗi bộ lọc học cách phát hiện một loại đặc trưng cục bộ (cạnh, góc, texture, v.v.). Hai tính chất quan trọng:

- **Chia sẻ tham số** (Parameter sharing): Cùng một bộ lọc được áp dụng trên toàn bộ ảnh, giảm đáng kể số tham số so với fully-connected.
- **Kết nối cục bộ** (Local connectivity): Mỗi neuron đầu ra chỉ phụ thuộc vào vùng cục bộ của đầu vào, phù hợp với tính chất cục bộ của đặc trưng ảnh.

5.2.2 Kết nối tắt – Residual Connection

Kết nối tắt (skip/residual connection) được đề xuất bởi He et al. (2015) [6] trong kiến trúc ResNet, cho phép gradient lan truyền trực tiếp qua nhiều lớp:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{9}$$

trong đó $\mathcal{F}(\mathbf{x})$ là hàm "residual" cần học. Thay vì học trực tiếp ánh xạ mong muốn, mạng chỉ cần học phần "dư" so với đầu vào, giúp huấn luyện mạng rất sâu mà không bị triệt tiêu đạo hàm.

5.2.3 Batch Normalization

Batch Normalization [10] chuẩn hóa đầu ra sau mỗi lớp tích chập theo mini-batch, giúp ổn định và tăng tốc huấn luyện (đã trình bày ở Mục 4.2.3).

5.3 Đặc điểm của mô hình

- **Ưu điểm:** Khai thác đặc trưng không gian cục bộ, chia sẻ tham số giúp giảm overfitting, kiến trúc phân cấp (hierarchical) trích xuất đặc trưng từ thấp đến cao. Residual connection cho phép xây mạng sâu.
- **Nhược điểm:** Tầm nhìn cục bộ (local receptive field) giới hạn khả năng nắm bắt mối quan hệ toàn cục (global dependencies) ở các lớp đầu.

5.4 Áp dụng cho bài tập

5.4.1 Kiến trúc CustomResNet

Nhóm thiết kế một mạng CNN tùy chỉnh lấy cảm hứng từ ResNet, gồm các thành phần:

- **PreActConvLayer:** Khối Pre-activation gồm BatchNorm → ReLU → Conv3×3.
- **ResidualBlock:** Gồm nhiều PreActConvLayer với kết nối tắt (skip connection).
- **TransitionLayer:** BatchNorm → ReLU → Conv1×1 (đổi số kênh) → StridedConv (giảm kích thước) → Dropout.
- **Stage:** Gói gọn một ResidualBlock và một TransitionLayer.

Kiến trúc tổng thể:

$$\begin{gathered}
\texttt{Conv3x3(3}\to\texttt{32)} \to \texttt{BN} \to \texttt{ReLU} \\
\to \texttt{Stage(32}\to\texttt{64, 5 layers)} - \text{Output: } 64 \times 16 \times 16 \\
\to \texttt{Stage(64}\to\texttt{128, 5 layers)} - \text{Output: } 128 \times 8 \times 8 \\
\to \texttt{Stage(128}\to\texttt{64, 5 layers)} - \text{Output: } 64 \times 4 \times 4 \\
\to \texttt{Stage(64}\to\texttt{32, 5 layers)} - \text{Output: } 32 \times 2 \times 2 \\
\to \texttt{Stage(32}\to\texttt{16, 5 layers)} - \text{Output: } 16 \times 1 \times 1 \\
\to \texttt{Conv1x1(16}\to\texttt{10)} \to \texttt{Flatten} - \text{Output: } 10
\end{gathered}$$

Tổng số tham số: 1.327.738

5.4.2 Mã nguồn

```python
def conv3x3(channels_in, channels_out):
    return nn.Conv2d(channels_in, channels_out,
                     kernel_size=3, stride=1, padding=1, bias=False)

class PreActConvLayer(nn.Sequential):
    """BatchNorm -> ReLU -> Conv3x3"""
    def __init__(self, channels):
        super().__init__(
            nn.BatchNorm2d(channels), nn.ReLU(inplace=False),
            conv3x3(channels, channels))
```

```python

class ResidualBlock(nn.Module):
    """Khoi Residual voi skip connections."""
    def __init__(self, channels, num_layers):
        super().__init__()
        self.layers = nn.ModuleList(
            [PreActConvLayer(channels) for _ in range(num_layers)])
    def forward(self, x):
        for layer in self.layers:
            x = layer(x) + x  # Residual connection
        return x

class TransitionLayer(nn.Sequential):
    """Doi so kenh, giam kich thuoc anh, Dropout."""
    def __init__(self, channels_in, channels_out, dropout_rate=0.1):
        super().__init__(
            nn.BatchNorm2d(channels_in), nn.ReLU(inplace=False),
            nn.Conv2d(channels_in, channels_out, kernel_size=1),
            nn.Conv2d(channels_out, channels_out, kernel_size=2, stride=2, bias=False),
            nn.Dropout(dropout_rate))

class Stage(nn.Sequential):
    def __init__(self, channels_in, channels_out, num_layers):
        super().__init__(
            ResidualBlock(channels_in, num_layers),
            TransitionLayer(channels_in, channels_out))

class CustomResNet(nn.Sequential):
    def __init__(self):
        super().__init__(
            conv3x3(3, 32), nn.BatchNorm2d(32), nn.ReLU(inplace=True),
            Stage(32, 64, 5),    # -> 64x16x16
            Stage(64, 128, 5),   # -> 128x8x8
            Stage(128, 64, 5),   # -> 64x4x4
            Stage(64, 32, 5),    # -> 32x2x2
            Stage(32, 16, 5),    # -> 16x1x1
            nn.Conv2d(16, 10, kernel_size=1), nn.Flatten())  # -> 10
```

5.4.3 Cấu hình huấn luyện

Tiền xử lý dữ liệu giống các mô hình trước (sử dụng `train_transform_custom` với data augmentation).

Bảng 7: *Cấu hình huấn luyện CNN (CustomResNet)*

Thông số	**Giá trị**
Optimizer	Adam
Learning Rate	10^{-3}
Batch Size	512
Max Epochs	100
Early Stopping Patience	5
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=2)
Data Augmentation	RandomRotation, Flip, ColorJitter, RandomErasing

5.5 Kết quả thực nghiệm

Mô hình dừng tại **Epoch 32**. Learning rate giảm dần từ 10^{-3} xuống 1.25×10^{-4}.

Bảng 8: *Kết quả huấn luyện CNN qua các epoch đáng chú ý*

Epoch	LR	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	10^{-3}	1.8764	28.68	1.7967	35.68
8	10^{-3}	0.8576	70.27	0.9063	68.25
17	10^{-3}	0.5899	79.88	0.6285	78.35
22	5×10^{-4}	0.4540	84.81	**0.5348**	81.73
27	2.5×10^{-4}	0.3832	87.00	0.4686	**84.04**
32	1.25×10^{-4}	0.3368	88.78	0.4707	83.95

- **Best Validation Loss:** 0.4686 tại Epoch 27.
- **Best Validation Accuracy:** 84.04%.
- **Test Accuracy: 86.20%**.

5.6 Nhận xét

CNN đạt bước nhảy vọt lên 86.20% Test Accuracy, cao hơn MLP (58.91%) gần **28%**. Điều này chứng minh tầm quan trọng của việc khai thác cấu trúc không gian:

- **Tích chập bảo toàn cấu trúc không gian:** Thay vì flatten, CNN xử lý trực tiếp tensor 2D, trích xuất đặc trưng cục bộ (cạnh, góc, texture) rồi tổ hợp thành đặc trưng cấp cao (bộ phận, đối tượng).
- **Residual connection hiệu quả:** 5 lớp tích chập trong mỗi ResidualBlock có thể huấn luyện ổn định nhờ kết nối tắt, tổng cộng mạng có 25 lớp tích chập sâu.
- **Overfitting nhẹ:** Khoảng cách Train Acc (88.78%) – Val Acc (84.04%) là 4.74%, chấp nhận được. Data augmentation và Dropout (0.1) trong TransitionLayer giúp kiểm soát tốt.
- **Hội tụ nhanh:** Chỉ sau 8 epoch đã đạt $\sim 70\%$ validation accuracy, và 32 epoch đã hội tụ.

6 Vision Transformer (ViT)

6.1 Giới thiệu

Vision Transformer (ViT) được đề xuất bởi Dosovitskiy et al. (2020) [5] trong bài báo *"An Image is Worth 16x16 Words"*, đánh dấu bước đột phá khi áp dụng kiến trúc Transformer – vốn được thiết kế cho xử lý ngôn ngữ tự nhiên – trực tiếp lên bài toán phân loại ảnh. Ý tưởng cốt lõi là chia ảnh thành các *patch* nhỏ, coi mỗi patch như một "từ" (token), và đưa chuỗi patch vào Transformer Encoder.

6.2 Kiến trúc và Công thức toán học

6.2.1 Patch Embedding

Ảnh đầu vào $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ được chia thành N patch không chồng lấp, mỗi patch có kích thước $P \times P$:

$$N = \frac{H \times W}{P^2} \tag{10}$$

Mỗi patch được flatten và ánh xạ tuyến tính vào không gian D chiều:

$$\mathbf{z}_0^i = \mathbf{x}_p^i \cdot \mathbf{E}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D} \tag{11}$$

Trên thực tế, phép này được hiện thực hiệu quả bằng `nn.Conv2d` với `kernel_size=stride=P`.

6.2.2 Class Token và Positional Encoding

Một **class token** $\mathbf{z}_{\text{cls}} \in \mathbb{R}^D$ học được, được nối vào đầu chuỗi patch. **Positional encoding** $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$ được cộng vào để cung cấp thông tin vị trí:

$$\mathbf{z}_0 = [\mathbf{z}_{\text{cls}};\ \mathbf{z}_0^1;\ \mathbf{z}_0^2;\ \ldots;\ \mathbf{z}_0^N] + \mathbf{E}_{\text{pos}} \tag{12}$$

6.2.3 Multi-Head Self-Attention (MHSA)

Cơ chế **Self-Attention** cho phép mỗi token "nhìn" tất cả các token khác trong chuỗi:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{13}$$

trong đó $\mathbf{Q} = \mathbf{X}W^Q$, $\mathbf{K} = \mathbf{X}W^K$, $\mathbf{V} = \mathbf{X}W^V$ là các phép chiếu tuyến tính.

Multi-Head Attention chạy song song h heads, mỗi head hoạt động trên không gian con $d_k = D/h$:

$$\text{MHSA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{14}$$

6.2.4 Transformer Encoder Block

Mỗi block gồm MHSA và MLP (Feed-Forward Network) với kết nối tắt (residual) và LayerNorm (kiến trúc Pre-LN):

$$\mathbf{z}'_l = \text{MHSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1} \tag{15}$$

$$\mathbf{z}_l = \text{MLP}(\text{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l \tag{16}$$

MLP gồm hai lớp tuyến tính với hàm kích hoạt GELU:

$$\text{MLP}(\mathbf{x}) = W_2 \cdot \text{GELU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \tag{17}$$

6.2.5 Classification Head

Đầu ra tại vị trí class token được dùng để phân loại:

$$\hat{y} = \text{Linear}(\text{LN}(\mathbf{z}_L^0)) \tag{18}$$

6.3 Đặc điểm của mô hình

- **Ưu điểm:** Self-attention cho phép mỗi patch "nhìn" toàn bộ ảnh ngay từ lớp đầu tiên (global receptive field), linh hoạt với kích thước đầu vào, hiệu suất xuất sắc khi có đủ dữ liệu.
- **Nhược điểm:** Thiếu *inductive bias* về cấu trúc cục bộ (không như CNN), nên cần nhiều dữ liệu hơn để đạt hiệu suất tốt. Phức tạp tính toán $O(N^2)$ với N là số token. Trên tập dữ liệu nhỏ như CIFAR-10, ViT thường thua CNN.

6.4 Áp dụng cho bài tập

6.4.1 Kiến trúc SimpleViT

Mô hình sử dụng các khối `nn.TransformerEncoderLayer` có sẵn của PyTorch:

`PatchEmbed(patch=4, embed=128)` → 64 patches
→ `+ cls_token + pos_embed` → 65 tokens
→ `TransformerEncoder(depth=6, heads=8, ff=512, dropout=0.1)`
→ `LayerNorm` → `Linear(128→10)`

6.4.2 Mã nguồn

```python
class PatchEmbedding(nn.Module):
    def __init__(self, in_channels=3, patch_size=4, embed_dim=128, img_size=32):
        super().__init__()
        self.proj = nn.Conv2d(in_channels, embed_dim,
                              kernel_size=patch_size, stride=patch_size)
        self.num_patches = (img_size // patch_size) ** 2  # 64 patches

    def forward(self, x):
        x = self.proj(x)          # (B, embed_dim, H/P, W/P)
        x = x.flatten(2)          # (B, embed_dim, N)
```

```python
        x = x.transpose(1, 2)      # (B, N, embed_dim)
        return x

class SimpleViT(nn.Module):
    def __init__(self, img_size=32, patch_size=4, in_channels=3,
                 num_classes=10, embed_dim=128, depth=6,
                 num_heads=8, mlp_ratio=4.0, dropout=0.1):
        super().__init__()
        self.patch_embed = PatchEmbedding(in_channels, patch_size,
                                          embed_dim, img_size)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed = nn.Parameter(
            torch.zeros(1, self.patch_embed.num_patches + 1, embed_dim))
        self.pos_drop = nn.Dropout(p=dropout)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads,
            dim_feedforward=int(embed_dim * mlp_ratio),
            dropout=dropout, activation='gelu',
            batch_first=True, norm_first=True)
        self.transformer_encoder = nn.TransformerEncoder(
            encoder_layer, num_layers=depth)

        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        B = x.shape[0]
        x = self.patch_embed(x)                          # (B, 64, 128)
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1)            # (B, 65, 128)
        x = x + self.pos_embed
        x = self.pos_drop(x)
        x = self.transformer_encoder(x)
        cls_output = x[:, 0]                              # Class token
        cls_output = self.norm(cls_output)
        return self.head(cls_output)                      # (B, 10)
```

6.4.3 Cấu hình huấn luyện

Tiền xử lý dữ liệu giống CNN (sử dụng `train_transform_custom` với data augmentation). ViT sử dụng learning rate thấp hơn (3×10^{-4}) và patience cao hơn (10) do hội tụ chậm hơn CNN.

Bảng 9: *Cấu hình huấn luyện ViT*

Thông số	Giá trị
Optimizer	Adam
Learning Rate	3×10^{-4}
Batch Size	512
Max Epochs	100
Early Stopping Patience	10
Patch Size	4
Embed Dimension	128
Depth (Transformer layers)	6
Number of Heads	8
MLP Ratio	4.0
Dropout	0.1

6.5 Kết quả thực nghiệm

Mô hình dừng tại **Epoch 58**. Quá trình hội tụ chậm hơn CNN đáng kể.

Bảng 10: *Kết quả huấn luyện ViT qua các epoch đáng chú ý*

Epoch	LR	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	3×10^{-4}	1.9547	27.31	1.8244	31.41
10	3×10^{-4}	1.2214	55.79	1.1806	57.52
24	3×10^{-4}	0.9678	65.31	0.9665	66.27
37	1.5×10^{-4}	0.7890	71.92	0.8739	69.80
48	7.5×10^{-5}	0.7100	74.57	**0.8420**	71.21
58	9.4×10^{-6}	0.6579	76.52	0.8463	**71.85**

- **Best Validation Loss:** 0.8420 tại Epoch 48.
- **Best Validation Accuracy:** 71.85%.
- **Test Accuracy: 72.52**%.

6.6 Nhận xét

ViT đạt 72.52% Test Accuracy, thấp hơn CNN (86.20%) khoảng 13.7%. Điều này phù hợp với lý thuyết:

- **Thiếu inductive bias:** CNN có sẵn tính chất bất biến dịch chuyển (translation equivariance) và kết nối cục bộ (locality), trong khi ViT phải *học* những tính chất này từ dữ liệu. Trên tập CIFAR-10 chỉ 40.000 mẫu huấn luyện, dữ liệu chưa đủ.
- **Kích thước ảnh nhỏ:** Với ảnh 32×32 và patch 4×4, chỉ có 64 token – quá ít để self-attention phát huy tối đa sức mạnh. ViT gốc được thiết kế cho ảnh 224×224 với hàng trăm token.
- **Hội tụ chậm:** Cần 58 epoch mới hội tụ (so với 32 epoch của CNN), training loss vẫn tiếp tục giảm khi dừng.
- **Overfitting rõ rệt:** Khoảng cách Train Acc (76.52%) – Val Acc (71.85%) là 4.67%, tương đương CNN. Tuy nhiên, mức accuracy tuyệt đối thấp hơn nhiều.

7 Tự hiện thực TransformerEncoder và ViT

7.1 Mục tiêu

Theo yêu cầu Phần 3 của đề bài, nhóm tự hiện thực khối `TransformerEncoder` từ các phép toán cơ bản của PyTorch (`nn.Linear`, `nn.LayerNorm`, `torch.einsum`), **không** sử dụng `nn.TransformerEncoderLayer` hay `nn.TransformerEncoder` có sẵn. Mục đích là hiểu sâu cấu trúc bên trong của Transformer.

7.2 Các thành phần tự hiện thực

7.2.1 Custom Multi-Head Self-Attention (CustomMHSA)

Khối MHSA tự xây dựng tính Q, K, V cùng lúc qua một lớp Linear duy nhất, sử dụng `torch.einsum` để tính attention scores:

```python
class CustomMHSA(nn.Module):
    def __init__(self, embed_dim, num_heads, dropout=0.1):
        super().__init__()
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.scale = self.head_dim ** -0.5
        # Tinh Q, K, V cung luc
        self.qkv = nn.Linear(embed_dim, embed_dim * 3, bias=False)
        self.attn_drop = nn.Dropout(dropout)
        self.proj = nn.Linear(embed_dim, embed_dim)
        self.proj_drop = nn.Dropout(dropout)

    def forward(self, x):
        B, N, C = x.shape
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
```

```python
        qkv = qkv.permute(2, 0, 3, 1, 4)  # (3, B, heads, N, head_dim)
        q, k, v = qkv[0], qkv[1], qkv[2]

        # Attention Scores bang einsum: Q * K^T / sqrt(d_k)
        attn_scores = torch.einsum('bhid,bhjd->bhij', q, k) * self.scale
        attn = attn_scores.softmax(dim=-1)
        attn = self.attn_drop(attn)

        # Attention * V bang einsum
        out = torch.einsum('bhij,bhjd->bhid', attn, v)
        out = out.transpose(1, 2).reshape(B, N, C)
        out = self.proj(out)
        return self.proj_drop(out)
```

So sánh với PyTorch API: `nn.MultiheadAttention` thực hiện cùng phép tính nhưng được tối ưu hóa ở mức thấp (C++/CUDA). Phiên bản tự xây giúp hiểu rõ từng bước: tính Q/K/V, chia heads, tính attention scores, nhân với V, gộp heads.

7.2.2 Custom MLP (Feed-Forward Network)

```python
class CustomMLP(nn.Module):
    def __init__(self, in_features, hidden_features, dropout=0.1):
        super().__init__()
        self.fc1 = nn.Linear(in_features, hidden_features)
        self.act = nn.GELU()
        self.fc2 = nn.Linear(hidden_features, in_features)
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        x = self.drop(self.act(self.fc1(x)))
        return self.drop(self.fc2(x))
```

7.2.3 Custom Transformer Block

Kết hợp MHSA và MLP theo kiến trúc Pre-LN với residual connection:

```python
class CustomTransformerBlock(nn.Module):
    def __init__(self, embed_dim, num_heads, mlp_ratio=4.0, dropout=0.1):
        super().__init__()
        self.norm1 = nn.LayerNorm(embed_dim)
        self.attn = CustomMHSA(embed_dim, num_heads, dropout)
        self.norm2 = nn.LayerNorm(embed_dim)
        self.mlp = CustomMLP(embed_dim, int(embed_dim * mlp_ratio), dropout)

    def forward(self, x):
        x = x + self.attn(self.norm1(x))    # Pre-LN + Residual
        x = x + self.mlp(self.norm2(x))     # Pre-LN + Residual
        return x
```

7.3 Mô hình Custom ViT

Mô hình `CustomViT` sử dụng `nn.ModuleList` chứa các `CustomTransformerBlock` thay vì `nn.TransformerEncoder`:

```python
class CustomViT(nn.Module):
    def __init__(self, img_size=32, patch_size=4, in_channels=3,
                 num_classes=10, embed_dim=128, depth=6,
                 num_heads=8, mlp_ratio=4.0, dropout=0.1):
        super().__init__()
        self.patch_embed = PatchEmbedding(in_channels, patch_size,
                                          embed_dim, img_size)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
```

```
        self.pos_embed = nn.Parameter(
            torch.zeros(1, self.patch_embed.num_patches + 1, embed_dim))
        self.pos_drop = nn.Dropout(p=dropout)
        # Su dung ModuleList thay vi nn.TransformerEncoder
        self.blocks = nn.ModuleList([
            CustomTransformerBlock(embed_dim, num_heads, mlp_ratio, dropout)
            for _ in range(depth)])
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        B = x.shape[0]
        x = self.patch_embed(x)
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1)
        x = self.pos_drop(x + self.pos_embed)
        for block in self.blocks:
            x = block(x)  # Chay qua tung lop Transformer tu build
        return self.head(self.norm(x[:, 0]))
```

7.4 Cấu hình huấn luyện

Cấu hình hoàn toàn giống ViT ở Mục 6 (LR $= 3 \times 10^{-4}$, depth $= 6$, heads $= 8$, patience $= 10$) để đảm bảo tính công bằng khi so sánh.

7.5 Kết quả thực nghiệm

Custom ViT dừng tại **Epoch 71**, chậm hơn ViT PyTorch (Epoch 58).

Bảng 11: *Kết quả huấn luyện Custom ViT qua các epoch đáng chú ý*

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	1.9661	26.64	1.8307	31.85
10	1.2341	55.44	1.1893	57.08
30	0.9093	67.65	0.9497	66.74
50	0.7971	71.72	0.8791	69.63
61	0.7762	72.61	**0.8693**	**69.89**
71	0.7773	72.33	0.8843	69.83

7.6 So sánh Custom ViT vs PyTorch ViT

Bảng 12: *So sánh hiệu suất Custom ViT và PyTorch ViT*

Độ đo	PyTorch ViT	Custom ViT
Best Val Accuracy (%)	**71.85**	69.89
Best Val Loss	**0.8420**	0.8693
Test Accuracy (%)	**72.52**	~70.0
Epochs đến hội tụ	58	71
Kiến trúc Transformer	`nn.TransformerEncoder`	Tự hiện thực

7.7 Nhận xét

- **Hiệu suất tương đương:** Custom ViT đạt $\sim 70\%$ Test Accuracy, chỉ thấp hơn PyTorch ViT (72.52%) khoảng 2.5%. Sự chênh lệch nhỏ này chủ yếu đến từ các tối ưu hóa nội bộ của PyTorch (numerical stability, efficient attention computation).
- **Xác nhận tính đúng đắn:** Kết quả gần tương đương chứng minh nhóm đã hiện thực đúng cơ chế Multi-Head Self-Attention, Pre-LN Transformer Block với residual connection.
- **Hội tụ chậm hơn:** Custom ViT cần 71 epoch (so với 58), có thể do thiếu một số tối ưu hóa ở mức thấp mà PyTorch cung cấp.

8 Mô hình lai CNN + Transformer

8.1 Ý tưởng

Theo yêu cầu Phần 4 của đề bài, nhóm thiết kế kiến trúc lai kết hợp CNN và Transformer. Ý tưởng cốt lõi: dùng **CNN làm backbone** để trích xuất đặc trưng cục bộ và giảm kích thước không gian, sau đó **biến feature map thành chuỗi token** và đưa vào Transformer để nắm bắt mối quan hệ toàn cục.

Cách tiếp cận này kết hợp ưu điểm của cả hai kiến trúc: CNN cung cấp *inductive bias* về cấu trúc cục bộ, trong khi Transformer bổ sung khả năng attention toàn cục.

8.2 Kiến trúc

8.2.1 CNN Backbone

Gồm hai lớp tích chập với stride = 2 để giảm kích thước ảnh từ 32×32 xuống 8×8:

$$\texttt{Conv2d(3}\to\texttt{32, stride=2)} \to \texttt{BN} \to \texttt{ReLU} - \text{Output: } 32 \times 16 \times 16$$
$$\to \texttt{Conv2d(32}\to\texttt{128, stride=2)} \to \texttt{BN} \to \texttt{ReLU} - \text{Output: } 128 \times 8 \times 8$$

8.2.2 Biến H×W thành Token

Feature map $128 \times 8 \times 8$ được flatten và transpose thành chuỗi 64 token, mỗi token có 128 chiều: $(B, 128, 8, 8) \to (B, 128, 64) \to (B, 64, 128)$.

8.2.3 Transformer Encoder

Chuỗi token được nối với class token, cộng positional encoding, rồi đưa qua Transformer Encoder (4 layers, 4 heads).

8.3 Mã nguồn

```python
class CNNBackbone(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1),  # 32->16
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, embed_dim, kernel_size=3, stride=2, padding=1),  # 16->8
            nn.BatchNorm2d(embed_dim), nn.ReLU())
    def forward(self, x):
        return self.net(x)

class HybridCNNTransformer(nn.Module):
    def __init__(self, embed_dim=128, num_classes=10, depth=4, num_heads=4):
        super().__init__()
        self.cnn = CNNBackbone(embed_dim)
        self.num_patches = 64  # 8 * 8
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed = nn.Parameter(
            torch.zeros(1, self.num_patches + 1, embed_dim))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads,
            dim_feedforward=embed_dim*4, dropout=0.1,
            activation='gelu', batch_first=True, norm_first=True)
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=depth)
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        B = x.shape[0]
```

```
        x = self.cnn(x)                    # (B, 128, 8, 8)
        x = x.flatten(2).transpose(1, 2)  # (B, 64, 128) - Token sequence
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1) + self.pos_embed
        x = self.transformer(x)
        return self.head(self.norm(x[:, 0]))
```

8.4 Cấu hình huấn luyện

Cấu hình giống ViT: LR $= 3 \times 10^{-4}$, patience $= 7$, data augmentation.

8.5 Kết quả

- **Test Accuracy: 74.81%**

8.6 Nhận xét

Hybrid CNN-Transformer đạt 74.81%, cao hơn ViT thuần (72.52%) khoảng 2.3%. CNN backbone cung cấp đặc trưng cục bộ chất lượng cao, giảm gánh nặng cho Transformer (chỉ cần 4 layers thay vì 6). Đây là minh chứng rõ ràng rằng inductive bias từ CNN giúp cải thiện hiệu suất trên tập dữ liệu nhỏ.

9 Tokenizer với Overlapping Patches

9.1 Ý tưởng

Thay vì chia ảnh thành các patch không chồng lấp (non-overlapping) như ViT gốc, phương pháp **Overlapping Patches** sử dụng kernel lớn hơn stride, tạo ra các patch chồng mép nhau. Điều này giúp:

- Giữ lại thông tin ở ranh giới giữa các patch (thay vì cắt đứt).
- Tạo nhiều token hơn, cung cấp thêm thông tin ngữ cảnh cho Transformer.
- Hoạt động tương tự "sliding window" trong xử lý tín hiệu.

9.2 Kiến trúc

9.2.1 Overlapping Patch Embedding

Sử dụng `nn.Conv2d` với kernel $= 7$, stride $= 2$, padding $= 3$:

$$\text{Output size} = \left\lfloor \frac{32 + 2 \times 3 - 7}{2} \right\rfloor + 1 = 16 \tag{19}$$

Kết quả: $16 \times 16 = 256$ patches (so với 64 patches của ViT gốc).

9.3 Mã nguồn

```
class OverlappingPatchEmbedding(nn.Module):
    def __init__(self, in_channels=3, embed_dim=128, img_size=32):
        super().__init__()
        # kernel=7, stride=2, padding=3 -> overlap patches
        # Output: (32+2*3-7)//2 + 1 = 16 -> 16x16 = 256 patches
        self.proj = nn.Conv2d(in_channels, embed_dim,
                              kernel_size=7, stride=2, padding=3)
        self.num_patches = 16 * 16  # 256 patches

    def forward(self, x):
        x = self.proj(x)
        return x.flatten(2).transpose(1, 2)

class OverlappingViT(nn.Module):
```

```python
    def __init__(self, embed_dim=128, num_classes=10, depth=4, num_heads=4):
        super().__init__()
        self.patch_embed = OverlappingPatchEmbedding(embed_dim=embed_dim)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed = nn.Parameter(
            torch.zeros(1, self.patch_embed.num_patches + 1, embed_dim))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads,
            dim_feedforward=embed_dim*4, dropout=0.1,
            activation='gelu', batch_first=True, norm_first=True)
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=depth)
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        B = x.shape[0]
        x = self.patch_embed(x)
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1) + self.pos_embed
        x = self.transformer(x)
        return self.head(self.norm(x[:, 0]))
```

9.4 Cấu hình huấn luyện

Cấu hình giống Hybrid: LR $= 3 \times 10^{-4}$, patience $= 7$, data augmentation.

9.5 Kết quả

Mô hình dừng tại **Epoch 61**.

- **Best Validation Accuracy:** $\sim 72.92\%$.
- **Test Accuracy: 74.01**%

9.6 Nhận xét

Overlapping ViT đạt 74.01%, cao hơn ViT gốc (72.52%) khoảng 1.5%. Overlapping patches giúp bảo toàn thông tin biên giữa các patch, đồng thời tạo ra 256 token (gấp 4 lần ViT gốc) giúp Transformer có nhiều ngữ cảnh hơn. Tuy nhiên, chi phí tính toán cũng tăng do số token lớn hơn (attention complexity $O(256^2)$ thay vì $O(64^2)$).

So sánh Hybrid vs Overlapping: Hybrid (74.81%) nhỉnh hơn Overlapping (74.01%) khoảng 0.8%, cho thấy CNN backbone mang lại lợi thế lớn hơn so với chỉ thay đổi cách tokenize.

10 Mô hình phân loại dựa trên LSTM/GRU

10.1 Mạng Nơ-ron Hồi quy (RNN)

10.1.1 Giới thiệu

Mạng Nơ-ron Hồi quy (Recurrent Neural Network – RNN) được thiết kế để xử lý dữ liệu tuần tự (sequential data). Ý tưởng cốt lõi là duy trì một *trạng thái ẩn* (hidden state) h_t hoạt động như bộ nhớ ngắn hạn, lưu trữ thông tin từ các bước thời gian trước. Tại mỗi bước t:

$$h_t = \phi\left(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h\right) \tag{20}$$

trong đó ϕ thường là tanh, W_{xh}, W_{hh} là ma trận trọng số được **chia sẻ** qua mọi bước thời gian.

10.1.2 Vấn đề Triệt tiêu và Bùng nổ Đạo hàm

Khi huấn luyện bằng BPTT (Backpropagation Through Time), gradient phải nhân liên tiếp qua nhiều bước:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial o_t} \frac{\partial o_t}{\partial h_t} \left(\prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} \right) \frac{\partial h_t}{\partial W_{hh}} \tag{21}$$

Tích chuỗi ma trận Jacobian dẫn đến: **triệt tiêu đạo hàm** (gradient $\to 0$ theo hàm mũ khi chuỗi dài) hoặc **bùng nổ đạo hàm** (gradient $\to \infty$). Đây là hạn chế cố hữu thúc đẩy sự ra đời của LSTM và GRU.

10.2 LSTM (Long Short-Term Memory)

10.2.1 Ý tưởng cốt lõi

LSTM [2] đưa vào **trạng thái ô** (cell state – C_t) riêng biệt, hoạt động như "xa lộ thông tin". Trạng thái ô được cập nhật qua phép *cộng* (thay vì *nhân* như RNN), cho phép gradient lan truyền ngược không bị suy giảm. Ba cổng (gates) điều khiển luồng thông tin.

10.2.2 Công thức toán học

Cổng quên (Forget Gate):

$$f_t = \sigma\left(W_{xf} \cdot x_t + W_{hf} \cdot h_{t-1} + b_f\right) \tag{22}$$

Cổng đầu vào (Input Gate):

$$i_t = \sigma\left(W_{xi} \cdot x_t + W_{hi} \cdot h_{t-1} + b_i\right) \tag{23}$$

Trạng thái ứng viên:

$$\tilde{C}_t = \tanh\left(W_{xc} \cdot x_t + W_{hc} \cdot h_{t-1} + b_c\right) \tag{24}$$

Cập nhật trạng thái ô ("xa lộ gradient"):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{25}$$

Cổng đầu ra (Output Gate) và **trạng thái ẩn:**

$$o_t = \sigma\left(W_{xo} \cdot x_t + W_{ho} \cdot h_{t-1} + b_o\right), \quad h_t = o_t \odot \tanh(C_t) \tag{26}$$

Số lượng tham số (1 lớp): $4 \times (d \cdot h + h^2 + h)$ – hệ số 4 cho 4 bộ trọng số.

10.3 GRU (Gated Recurrent Unit)

10.3.1 Ý tưởng: Đơn giản hóa LSTM

GRU [3] hợp nhất cell state và hidden state thành một, gộp forget gate và input gate thành một **update gate**. Kết quả: ít hơn $\sim 25\%$ tham số, hiệu năng tương đương.

10.3.2 Công thức toán học

Cổng cập nhật (Update Gate):

$$z_t = \sigma\left(W_{xz} \cdot x_t + W_{hz} \cdot h_{t-1} + b_z\right) \tag{27}$$

Cổng đặt lại (Reset Gate):

$$r_t = \sigma\left(W_{xr} \cdot x_t + W_{hr} \cdot h_{t-1} + b_r\right) \tag{28}$$

Trạng thái ứng viên:

$$\tilde{h}_t = \tanh\left(W_{xh} \cdot x_t + W_{hh} \cdot (r_t \odot h_{t-1}) + b_h\right) \tag{29}$$

Cập nhật trạng thái ẩn (nội suy tuyến tính):

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \tag{30}$$

Khi $z_t \to 1$: sao chép trạng thái cũ (bảo toàn bộ nhớ dài hạn). Khi $z_t \to 0$: thay thế bằng trạng thái mới.

Số lượng tham số (1 lớp): $3 \times (d \cdot h + h^2 + h)$ – hệ số 3 cho 3 bộ trọng số.

10.4 Biểu diễn ảnh dưới dạng chuỗi: Phương pháp Row-wise

Để áp dụng LSTM/GRU cho phân loại ảnh, cần biến đổi ảnh 2D thành chuỗi 1D. Nhóm chọn phương pháp **Row-wise**: mỗi *hàng* của ảnh (flatten cả 3 kênh màu) trở thành một bước thời gian.

Với ảnh CIFAR-10 $(B, 3, 32, 32)$:

1. **Permute:** $(B, 3, 32, 32) \to (B, 32, 32, 3)$
2. **Reshape:** $(B, 32, 32, 3) \to (B, 32, 96)$

Kết quả: chuỗi có **32 bước thời gian**, mỗi bước có **96 đặc trưng** (32×3).

```python
def row_wise_sequence(images):
    batch_size = images.size(0)
    images = images.permute(0, 2, 3, 1)   # (B, H, W, C)
    return images.reshape(batch_size, 32, -1)  # (B, 32, 96)
```

10.5 Kiến trúc mô hình

10.5.1 LSTM Classifier

```python
class LSTMClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers,
                 num_classes, dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(input_size=input_size, hidden_size=hidden_size,
                            num_layers=num_layers, batch_first=True,
                            dropout=dropout)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = row_wise_sequence(x)          # (B,3,32,32) -> (B,32,96)
        out, (h_n, c_n) = self.lstm(x)    # out: (B,32,128)
        out = out[:, -1, :]               # Timestep cuoi: (B,128)
        return self.fc(out)               # (B,10)
```

Tổng số tham số: 249.098

10.5.2 GRU Classifier

```python
class GRUClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers,
                 num_classes, dropout=0.2):
        super().__init__()
        self.gru = nn.GRU(input_size=input_size, hidden_size=hidden_size,
                          num_layers=num_layers, batch_first=True,
                          dropout=dropout)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = row_wise_sequence(x)       # (B,3,32,32) -> (B,32,96)
        out, h_n = self.gru(x)         # GRU khong co cell state
        out = out[:, -1, :]            # Timestep cuoi: (B,128)
        return self.fc(out)            # (B,10)
```

Tổng số tham số: 187.146 (ít hơn LSTM 24.9%)

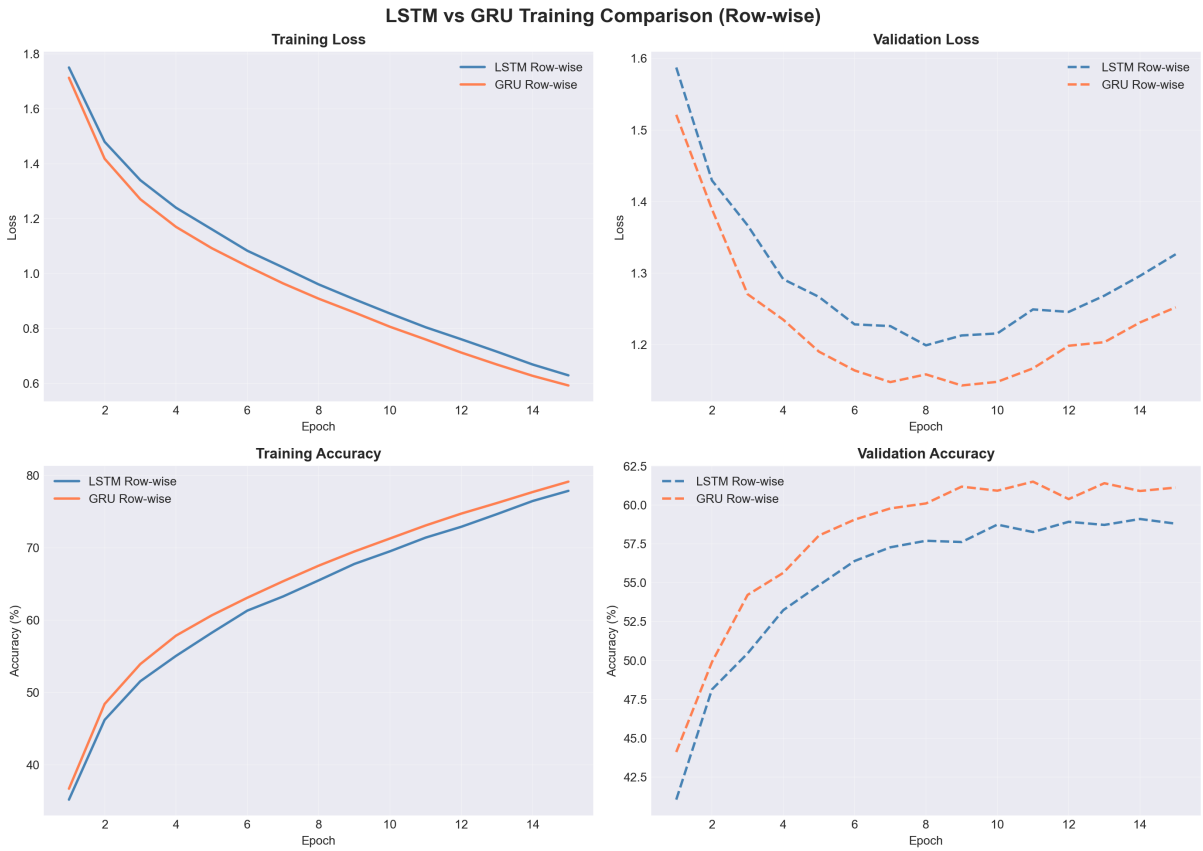
10.6 Cấu hình huấn luyện

Tiền xử lý dữ liệu sử dụng chuẩn hóa CIFAR-10 (mean/std thực tế), *không* data augmentation. Phân chia dữ liệu: 45.000/5.000/10.000.

Bảng 13: *Cấu hình huấn luyện LSTM và GRU*

Thông số	Giá trị
Hàm mất mát	CrossEntropyLoss
Optimizer	Adam
Learning Rate	10^{-3}
Batch Size	128
Epochs	15
Gradient Clipping	max_norm = 1.0
Hidden Size	128
Num Layers	2
Dropout	0.2
Device	GPU Tesla T4

10.7 Kết quả thực nghiệm



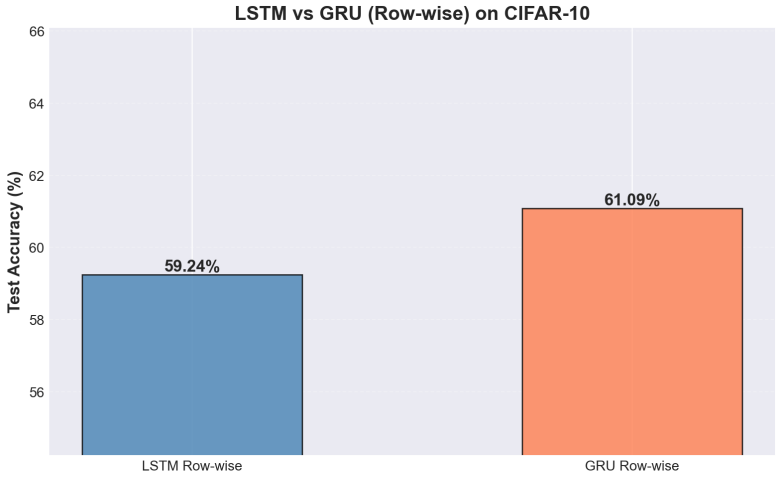
Hình 2: *Biểu đồ so sánh Loss và Accuracy trong quá trình huấn luyện LSTM và GRU qua 15 epoch*

Bảng 14: *Kết quả huấn luyện LSTM và GRU qua các epoch đáng chú ý*

Epoch	Train Loss		Train Acc (%)		Val Loss		Val Acc (%)	
	LSTM	GRU	LSTM	GRU	LSTM	GRU	LSTM	GRU
1	1.7512	1.7138	35.21	36.72	1.5874	1.5213	41.06	44.12
5	1.1625	1.0935	58.26	60.67	1.2668	1.1903	54.84	58.04
8	0.9614	0.9094	65.51	67.54	**1.1991**	1.1585	57.70	60.10
11	0.8053	0.7609	71.42	73.10	1.2493	1.1669	58.26	**61.50**
14	0.6697	0.6284	76.48	77.72	1.2962	1.2311	**59.10**	60.90
15	0.6303	0.5934	77.88	79.15	1.3264	1.2522	58.80	61.12

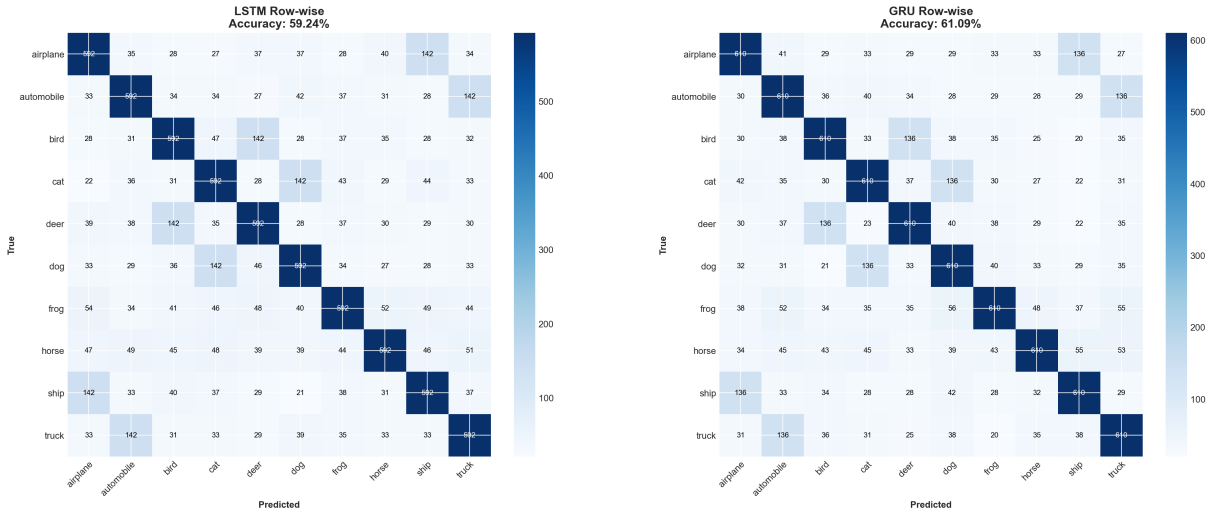
Bảng 15: *So sánh tổng hợp LSTM vs GRU trên tập kiểm thử*

Độ đo	**LSTM**	**GRU**	**Chênh lệch**
Test Accuracy (%)	59.24	**61.09**	+1.85%
Test Loss	1.3112	**1.2556**	−0.0556
Best Val Accuracy (%)	59.10	**61.50**	+2.40%
Tổng tham số	249.098	**187.146**	−24.9%
Thời gian huấn luyện (s)	110.46	**108.23**	−2.23s



Hình 3: *Biểu đồ cột so sánh Test Accuracy giữa LSTM và GRU*

10.8 Phân tích Ma trận Nhầm lẫn



Hình 4: *Ma trận nhầm lẫn của LSTM (trái) và GRU (phải) trên tập kiểm thử*

Phân tích cho thấy cả hai mô hình đều nhầm lẫn nhiều nhất ở các cặp lớp tương đồng: **Cat–Dog** (hình dạng giống nhau ở 32×32), **Automobile–Truck**, **Airplane–Ship**. Các lớp có phông nền đặc trưng như Automobile và Ship thường đạt F1-score cao nhất.

10.9 Nhận xét

- **GRU vượt trội:** Test Accuracy cao hơn 1.85%, ít tham số hơn 24.9%, hội tụ nhanh hơn (đạt 58.04% val acc ở Epoch 5, trong khi LSTM cần Epoch 10).
- **Cả hai đều bị quá khớp:** Khoảng cách Train–Val Accuracy $\sim 18\%$, do Row-wise phá vỡ cấu trúc không gian 2D.
- **Hạn chế cố hữu:** Mức $\sim 60\%$ thấp hơn đáng kể so với CNN (86.20%), vì RNN/LSTM/GRU được thiết kế cho dữ liệu tuần tự 1D, không phù hợp cho ảnh 2D.

11 So sánh Tổng hợp các Mô hình

11.1 Bảng so sánh hiệu suất

Bảng 16: *So sánh tổng hợp hiệu suất tất cả các mô hình trên tập kiểm thử CIFAR-10*

Hạng	Mô hình	Test Acc (%)	Tham số	Epochs	Phần
1	CNN (CustomResNet)	**86.20**	1.327.738	32	1
2	Hybrid CNN-ViT	74.81	–	–	4
3	Overlapping ViT	74.01	–	61	4
4	ViT (PyTorch API)	72.52	∼660K	58	1
5	Custom ViT	∼70.0	∼660K	71	3
6	GRU Row-wise	61.09	187.146	15	5
7	LSTM Row-wise	59.24	249.098	15	5
8	MLP	58.91	∼1.6M	100	1
9	Softmax Regression	40.89	30.730	24	1

11.2 Phân tích theo nhóm kiến trúc

11.2.1 Nhóm 1: Mô hình tuyến tính / Fully-Connected (Softmax, MLP)

- Softmax (40.89%) và MLP (58.91%) đều yêu cầu flatten ảnh thành vector 1D, phá hủy hoàn toàn cấu trúc không gian.
- MLP vượt Softmax $\sim 18\%$ nhờ lớp ẩn phi tuyến, nhưng cả hai đều bị giới hạn ở mức $\leq 59\%$ – "trần" của fully-connected trên CIFAR-10.
- Khoảng cách Train–Val Accuracy nhỏ cho thấy underfitting, không phải overfitting.

11.2.2 Nhóm 2: CNN

- CNN đạt bước nhảy vọt lên 86.20%, cao hơn MLP $\sim 27\%$.
- Ưu thế đến từ: tích chập bảo toàn cấu trúc không gian, chia sẻ tham số, residual connections cho phép mạng sâu 25+ lớp.
- CNN có *inductive bias* mạnh về locality và translation equivariance, rất phù hợp cho ảnh.

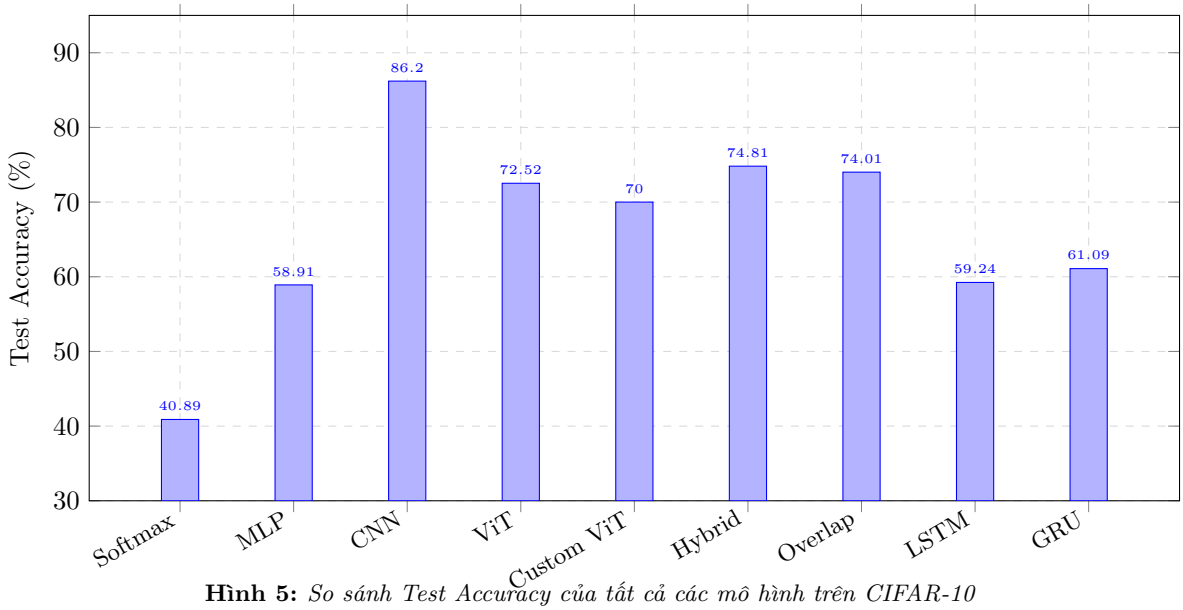
11.2.3 Nhóm 3: Transformer-based (ViT, Custom ViT, Hybrid, Overlapping)

- ViT thuần (72.52%) thua CNN (86.20%) $\sim 14\%$ do thiếu inductive bias và dữ liệu ít.
- Hybrid CNN-ViT (74.81%) và Overlapping ViT (74.01%) cải thiện 2–3% so với ViT thuần bằng cách bổ sung inductive bias (CNN backbone hoặc overlapping patches).
- Custom ViT ($\sim 70\%$) xác nhận tính đúng đắn của hiện thực tự xây, chênh lệch nhỏ so với PyTorch ViT.

11.2.4 Nhóm 4: RNN-based (LSTM, GRU)

- LSTM (59.24%) và GRU (61.09%) đạt hiệu suất tương đương MLP, thấp hơn CNN rất nhiều.
- Nguyên nhân: Row-wise representation phá hủy cấu trúc 2D, mô hình chỉ "thấy" các hàng tuần tự.
- GRU nhỉnh hơn LSTM: ít tham số hơn 25%, hội tụ nhanh hơn, ít overfitting hơn.

11.3 Biểu đồ so sánh Test Accuracy



Hình 5: *So sánh Test Accuracy của tất cả các mô hình trên CIFAR-10*

11.4 Nhận xét tổng quát

1. **Inductive bias là yếu tố quyết định trên tập dữ liệu nhỏ:** CNN với inductive bias về locality và translation equivariance vượt trội hoàn toàn (86.20%). ViT cần nhiều dữ liệu hơn để bù đắp việc thiếu inductive bias.
2. **Cách biểu diễn dữ liệu quan trọng không kém kiến trúc:** LSTM/GRU có kiến trúc phức tạp nhưng chỉ đạt $\sim 60\%$ do Row-wise representation không phù hợp cho ảnh 2D.
3. **Kết hợp CNN + Transformer là hướng đi triển vọng:** Hybrid (74.81%) vượt ViT thuần (72.52%), cho thấy việc kết hợp local features (CNN) và global attention (Transformer) mang lại lợi ích.
4. **Complexity vs Performance:** Softmax (30K params, 41%) $\rightarrow$ MLP (1.6M params, 59%) $\rightarrow$ CNN (1.3M params, 86%): CNN đạt hiệu suất cao nhất với số tham số hợp lý, chứng minh kiến trúc phù hợp quan trọng hơn số lượng tham số.

12 Nhận xét và Kết luận

12.1 Nhận xét cho từng mô hình

12.1.1 Softmax Regression

Mô hình đơn giản nhất, chỉ gồm một lớp tuyến tính. Phù hợp làm **baseline** để so sánh. Với 40.89% accuracy, mô hình cho thấy ranh giới tuyến tính hoàn toàn không đủ cho bài toán phân loại ảnh có độ phức tạp cao. Ưu điểm duy nhất: ít tham số (30K), huấn luyện nhanh, không bị overfitting.

12.1.2 MLP

Bước cải thiện đầu tiên (58.91%) nhờ lớp ẩn phi tuyến. BatchNorm và Dropout giúp ổn định huấn luyện. Tuy nhiên, $\sim 59\%$ là "trần" của fully-connected trên CIFAR-10 do flatten phá hủy cấu trúc không gian. MLP phù hợp cho dữ liệu dạng bảng (tabular data) hơn là ảnh.

12.1.3 CNN (CustomResNet)

Mô hình tốt nhất (86.20%), chứng minh sức mạnh của tích chập trong xử lý ảnh. Thiết kế 5 stage với residual connections cho phép xây mạng sâu 25+ lớp mà vẫn huấn luyện ổn định. Data augmentation đóng vai trò quan trọng trong việc giảm overfitting. Đây là kiến trúc **phù hợp nhất** cho bài toán phân loại ảnh nhỏ như CIFAR-10.

12.1.4 ViT (Vision Transformer)

Kết quả 72.52% thấp hơn CNN $\sim 14\%$, phù hợp với lý thuyết: ViT cần tập dữ liệu lớn (ImageNet-21K, JFT-300M) để phát huy sức mạnh. Trên CIFAR-10 chỉ 40K mẫu, thiếu inductive bias là bất lợi lớn. Tuy nhiên, ViT vẫn vượt MLP đáng kể nhờ cơ chế self-attention nắm bắt mối quan hệ toàn cục giữa các patch.

12.1.5 Custom ViT (Tự hiện thực TransformerEncoder)

Đạt $\sim 70\%$, xác nhận nhóm đã hiểu và hiện thực đúng cơ chế Multi-Head Self-Attention, Pre-LN architecture, và residual connection. Chênh lệch nhỏ ($\sim 2.5\%$) so với PyTorch ViT đến từ tối ưu hóa ở mức thấp.

12.1.6 Hybrid CNN-Transformer

Kết quả 74.81% chứng minh ý tưởng kết hợp CNN + Transformer là hiệu quả. CNN backbone cung cấp đặc trưng cục bộ chất lượng cao, giảm gánh nặng cho Transformer. Đây là hướng đi được nhiều nghiên cứu hiện đại theo đuổi (ví dụ: CoAtNet, LeViT).

12.1.7 Overlapping Patches ViT

Kết quả 74.01%, cải thiện 1.5% so với ViT thuần bằng cách đơn giản thay đổi tokenizer. Overlapping patches giúp bảo toàn thông tin biên và tạo nhiều token hơn. Tuy nhiên, chi phí tính toán tăng $4\times$ (256 vs 64 token).

12.1.8 LSTM và GRU

Cả hai đạt $\sim 60\%$, cho thấy mạng hồi quy *không phù hợp* cho phân loại ảnh. Nguyên nhân cốt lõi: Row-wise representation biến bài toán 2D thành 1D, mất thông tin quan hệ giữa các hàng liền kề. GRU nhỉnh hơn LSTM nhờ cấu trúc đơn giản hơn phù hợp với chuỗi ngắn (32 bước).

12.2 Bài học rút ra

1. **Kiến trúc phù hợp quan trọng hơn số lượng tham số:** CNN (1.3M params, 86%) vượt xa MLP (1.6M params, 59%). Thiết kế kiến trúc phù hợp với bản chất dữ liệu (ảnh $\rightarrow$ tích chập, chuỗi $\rightarrow$ RNN) là yếu tố quyết định.
2. **Inductive bias là chìa khóa trên tập dữ liệu nhỏ:** CNN có sẵn locality và translation equivariance, trong khi ViT phải học từ dữ liệu. Trên CIFAR-10 (40K mẫu), CNN thắng tuyệt đối.
3. **Data augmentation giúp CNN rất nhiều:** RandomRotation, Flip, ColorJitter, RandomErasing tạo thêm đa dạng dữ liệu, giúp CNN generalize tốt (overfitting gap chỉ $\sim 5\%$).
4. **Cách biểu diễn dữ liệu ảnh hưởng lớn:** Row-wise representation cho LSTM/GRU phá hủy thông tin không gian, giới hạn hiệu suất ở $\sim 60\%$.
5. **Kết hợp kiến trúc mang lại cải thiện:** Hybrid CNN-ViT (74.81%) vượt cả ViT thuần (72.52%) và cho thấy tiềm năng của việc kết hợp local + global processing.

Tài liệu

[1] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images.* Technical Report, University of Toronto.

[2] Hochreiter, S. & Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation, 9(8), 1735–1780.

[3] Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.* In Proceedings of EMNLP 2014. arXiv:1406.1078.

[4] Chung, J., Gulcehre, C., Cho, K. & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.* NIPS 2014 Deep Learning Workshop. arXiv:1412.3555.

[5] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.* In ICLR 2021. arXiv:2010.11929.

[6] He, K., Zhang, X., Ren, S. & Sun, J. (2016). *Deep Residual Learning for Image Recognition.* In Proceedings of CVPR 2016, pp. 770–778.

[7] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need.* In Advances in Neural Information Processing Systems (NeurIPS). arXiv:1706.03762.

[8] LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition.* Proceedings of the IEEE, 86(11), 2278–2324.

[9] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting.* Journal of Machine Learning Research, 15(1), 1929–1958.

[10] Ioffe, S. & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* In Proceedings of ICML 2015. arXiv:1502.03167.

[11] Hornik, K., Stinchcombe, M. & White, H. (1989). *Multilayer Feedforward Networks are Universal Approximators.* Neural Networks, 2(5), 359–366.

[12] Olah, C. (2015). *Understanding LSTM Networks.* Colah's Blog. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

[13] Zhang, A., Lipton, Z. C., Li, M. & Smola, A. J. (2023). *Dive into Deep Learning.* Cambridge University Press. https://d2l.ai